C 语言与计算思维
期末复习全攻略

摆渡 (Powered by GitHub Copilot)

2026 年 1 月 5 日

使用建议

本复习攻略旨在帮助大家高效梳理《C 语言与计算思维》课程的核心考点。

建议用法：请**快速浏览**全文，对于已经掌握的知识点一带而过，重点关注**自己不熟悉**或**容易混淆**的部分（如指针、文件操作、易错点辨析等）。

祝大家考试顺利，高分通过！

目录

1 计算机系统与原理 **4**
- 1.1 核心考点 4
- 1.2 简答题预测 4

2 算法基础 **4**
- 2.1 核心考点 4
- 2.2 简答题预测 5

3 实验与期末试卷分析 **5**
- 3.1 实验课核心内容总结 5
- 3.2 期末试卷高频考点解析 5

4 C 语言基础语法 **6**
- 4.1 数据类型与运算 6
- 4.2 输入输出 (Input/Output) 6
- 4.3 常量与宏定义 7
- 4.4 易错点 7

5 控制结构 **7**

6 数组与字符串 **7**
- 6.1 基本概念 7
- 6.2 常用字符串函数 (<string.h>) 8
- 6.3 其他重要函数 8

7 指针 (核心难点) **8**
- 7.1 指针基础与运算 8
- 7.2 指针与数组的深度辨析 9
- 7.3 复杂指针声明解析 9
- 7.4 简答题预测 10

8 函数与递归 **10**

9 结构体、共用体与枚举 **10**
- 9.1 核心考点 10
- 9.2 简答题预测 10

10 动态内存管理 (Dynamic Memory) **11**

10.1 栈 (Stack) 与堆 (Heap) 的区别 11
10.2 核心函数详解 11
10.3 常见错误与最佳实践 11

11 文件操作详解 (File I/O) 12
11.1 文件打开与关闭 12
11.2 文件打开模式 (Mode) 12
11.3 读写函数辨析 12
11.4 文件定位与随机读写 13
11.5 高频易错点 (坑) 13
11.6 预处理指令补充 14

12 核心算法详解与易错点深度剖析 14
12.1 排序算法 (Sorting) 14
12.1.1 冒泡排序 (Bubble Sort) 14
12.1.2 选择排序 (Selection Sort) 14
12.1.3 插入排序 (Insertion Sort) 15
12.2 查找算法 (Searching) 16
12.2.1 二分查找 (Binary Search) 16
12.3 数学与逻辑算法 17
12.3.1 辗转相除法 (GCD) 17
12.3.2 素数判断 (Prime Check) 17
12.4 字符串处理算法 17
12.4.1 字符串逆序 17
12.5 考试手写代码“救命”技巧 18

13 C 语言易混淆概念辨析 18

14 冷门与进阶考点拾遗 19

15 简答题预测汇总 20
15.1 计算机基础与算法 20
15.2 C 语言核心概念 21
15.3 常见算法特点与比较 21
15.4 实验课补充简答题 22

1 计算机系统与原理

1.1 核心考点

- **图灵测试 (Turing Test)**：测试机器是否具有人类智能的标准。如果测试者在不接触被测试者的情况下，无法区分人和机器，则认为机器通过测试。
- **计算机系统组成**：
  - **硬件 (Hardware)**：CPU、存储器、输入/输出设备（冯 • 诺依曼结构核心）。
  - **软件 (Software)**：系统软件（OS、驱动）、应用软件（Office、游戏）。
- **冯 · 诺依曼体系结构**：
  - **核心思想：存储程序** (Stored Program)。程序和数据都以二进制形式存储在存储器中。
  - **五大部件**：运算器、控制器、存储器、输入设备、输出设备。
- **数据表示**：
  - **二进制**：计算机内部使用二进制（0 和 1）。
  - **补码**：负数在计算机中以补码形式存储（按位取反 +1）。

1.2 简答题预测

Q: 什么是“存储程序”原理？

A: 将程序（指令序列）和数据一起存储在计算机的存储器中，计算机在运行时自动地从存储器中取出指令并执行，而不需要人工干预。

Q: 简述硬件与软件的关系。

A: 硬件是计算机的躯壳（物理实体），软件是计算机的灵魂（思想）。硬件提供物质基础，软件指挥硬件工作。两者相辅相成，缺一不可。

2 算法基础

2.1 核心考点

- **定义**：解决特定问题的一系列有限步骤。
- **五大特性**：
  1. **有穷性**：步骤有限。
  2. **确定性**：指令清晰无歧义。
  3. **输入**：0 个或多个。
  4. **输出**：1 个或多个。

5. **可行性**：每一步都能通过基本运算实现。

2.2 简答题预测

Q: 什么是算法？它有哪些基本特征？

A: 算法是解决特定问题的一系列明确、有限的指令。基本特征包括：有穷性、确定性、可行性、输入、输出。

3 实验与期末试卷分析

3.1 实验课核心内容总结

- **基础运算与日期**：
  - 模拟计算器：注意运算符优先级和除零错误。
  - 日期计算：闰年判断 `(year%4==0 && year%100!=0) || (year%400==0)`。
- **数组与结构体**：
  - 有序数组 vs 无序数组：插入、删除、查找的时间复杂度对比。
  - 结构体数组：如客户信息管理、歌唱比赛评分（去掉最高最低分求平均）。
- **递归应用**：
  - 汉诺塔：经典的递归分治问题。
  - 预测赢家：博弈论中的递归求解。
- **大数运算**：
  - 使用字符数组或整型数组模拟超大整数的加法，注意进位处理。
- **栈的应用**：
  - JSON 字符串括号匹配：利用栈的后进先出特性，遇到左括号入栈，遇到右括号出栈匹配。

3.2 期末试卷高频考点解析

- **选择题陷阱**：
  - **字符串长度**：`strlen` 遇到 `\0` 停止，转义字符（如 `\101`）算一个字符。
  - **逻辑短路**：`a || b++`，若 `a` 为真，`b++` 不执行。
  - **指针类型**：区分 `int *p[10]` (指针数组) 和 `int (*p)[10]` (数组指针)。
  - **作用域**：局部变量屏蔽同名全局变量。
- **程序填空题常考模式**：
  - **数学公式实现**：如海伦公式求三角形面积、泰勒级数求 `sin(x)`（注意循环终

止条件 `fabs(term) < 1e-5`）。
  - **最大公约数**：辗转相除法的 `while` 循环实现。
  - **条件判断**：三目运算符 `a>b?a:b` 的嵌套使用。
- **代码阅读题**：
  - 能够正确跟踪指针的移动和数组元素的访问。
  - 理解 `sizeof` 在不同情况下的值（数组名 vs 指针）。

4 C 语言基础语法

4.1 数据类型与运算

- **数据类型大小**：`sizeof` 是运算符。
  - `char` (1), `short` (2), `int` (4), `long` (4 或 8), `double` (8)。
- **类型转换**：
  - **隐式**：`int + double` → `double`。
  - **强制**：`(int)(3.14 + 0.9)` 结果是 4。
- **自增自减**：
  - `b = a++`：先赋值，后自增。
  - `b = ++a`：先自增，后赋值。
  - **真题陷阱**：`k=(n=b>a)||(m=a<b)`。注意逻辑短路，若前半部分为真，后半部分不执行。
- **位运算**：
  - `&`, `|`, `^`, `~`, `<<`, `>>`。
  - 技巧：`a << 1` (乘 2), `a >> 1` (除 2), `n & 1` (判断奇偶)。

4.2 输入输出 (Input/Output)

- **格式化输出 (printf)**：
  - `%d` (整型), `%f` (浮点), `%c` (字符), `%s` (字符串), `%p` (指针地址)。
  - **精度控制**：`%.2f` (保留 2 位小数)，`%5d` (宽 5 位右对齐)，`%-5d` (宽 5 位左对齐)。
- **格式化输入 (scanf)**：
  - **地址符**：除了字符串数组名外，变量前必须加 `&`。
  - **分隔符**：若格式串为 `"%d,%d"`，输入时必须加逗号。
  - **缓冲区残留**：混合输入字符和数字时，回车符可能被 `%c` 读取。建议在 `%c` 前

加空格：`scanf(" %c", &ch);`。

- **字符输入输出：**
  - `getchar()` / `putchar()`：一次只处理一个字符。

4.3 常量与宏定义

- **const 常量：**
  - `const int MAX = 100;`
  - 有类型检查，占用内存（通常在只读区）。
- **宏定义 (#define)：**
  - `#define MAX 100`
  - 预处理阶段文本替换，无类型检查，不占内存。
  - **注意：**宏定义末尾不要加分号。

4.4 易错点

- **除法陷阱：**`5 / 2 = 2`, `5.0 / 2 = 2.5`。
- **逻辑短路：**`A || B` (A 真 B 不执行), `A && B` (A 假 B 不执行)。
- **浮点比较：**严禁 `f == 0.0`，应用 `fabs(f) < 1e-6`。
- **赋值与相等：**`if(a=2)` 永远为真，且 `a` 变为 2。这是极高频考点。

5 控制结构

- **Switch-Case：**`case` 后必须是整型或字符型常量。注意**穿透性**（漏写 `break`）。
- **循环：**`continue` 跳过本次剩余，`break` 跳出循环。
- **易错点：**悬挂 `else`（与最近的 `if` 匹配）；分号错误（`if(x); ...`）。
- **真题模式：**利用 `do...while` 计算泰勒级数（如 `sin(x)`），注意循环终止条件通常是 `fabs(term) < 1e-5`。

6 数组与字符串

6.1 基本概念

- **定义：**字符串是以空字符 `'\0'` 结尾的字符数组。
- **初始化：**
  - `char s[] = "hello";` (自动加 `'\0'`，长度为 6，内容可变)

  - `char *p = "hello";` (字符串常量，通常存储在只读数据区，**不可修改**)
- **sizeof vs strlen：**
  - `sizeof`：计算数组占用的总字节数（含 `'\0'`）。
  - `strlen`：计算字符串的有效长度（不含 `'\0'`）。
  - **真题陷阱**：`strlen("abc\0def")` 结果是 3（遇到第一个 `\0` 停止）。

6.2 常用字符串函数 (<string.h>)

- **复制 (Copy)：**
  - `strcpy(dest, src)`：将 `src` 复制到 `dest`。
  - **危险**：若 `dest` 空间不足，会发生缓冲区溢出。
  - `strncpy(dest, src, n)`：最多复制 `n` 个字符。注意：若 `src` 长度 $> n$，则 `dest` 不会自动添加 `'\0'`，需手动添加。
- **拼接 (Concatenate)：**
  - `strcat(dest, src)`：将 `src` 拼接到 `dest` 后面。
  - **要求**：`dest` 必须有足够的剩余空间，且必须以 `'\0'` 结尾。
  - `strncat(dest, src, n)`：拼接前 `n` 个字符，会自动添加 `'\0'`。
- **比较 (Compare)：**
  - `strcmp(s1, s2)`：按字典序比较。
  - **返回值**：0 (相等), >0 (s1 大), <0 (s1 小)。
  - **陷阱**：严禁使用 `if(s1 == s2)` 比较内容（这是比较地址!）。
- **查找 (Search)：**
  - `strchr(s, c)`：查找字符 `c` 首次出现的位置（返回指针）。
  - `strstr(s1, s2)`：查找子串 `s2` 首次出现的位置。

6.3 其他重要函数

- `sprintf(buf, "%d", 123)`：将数据格式化输出到字符串中（常用于数字转字符串）。
- `atoi(s)` / `atof(s)`：字符串转整数/浮点数 (`<stdlib.h>`)。

7 指针 (核心难点)

7.1 指针基础与运算

- **定义与解引用：**

  - `int *p = &a;`：p 存储 a 的地址。
  - `*p`：访问 p 指向的内存单元（即 a）。
  - **口诀**：& 取地址，* 取值。
- **指针运算**：
  - `p + 1`：**不是地址加 1**，而是地址增加 sizeof(类型)。
  - `p1 - p2`：返回两指针之间的**元素个数**（不是字节数）。
  - **示例**：若 `int a[5]`，`p=a`，则 `*(p+2)` 等价于 `a[2]`。

7.2 指针与数组的深度辨析

- **访问方式等价性**：
  - `a[i]` $\iff$ `*(a+i)` $\iff$ `*(p+i)` $\iff$ `p[i]`。
- **本质区别**：
  - **数组名** a 是**常量指针**，不可修改（a++ 非法）。
  - **指针变量** p 是变量，可以修改指向（p++ 合法）。
- **二维数组指针**：
  - `int a[3][4];`
  - a：指向首行（类型 `int (*)[4]`）。
  - `a[0]`：指向首行首元素（类型 `int *`）。
  - `a[i][j]` $\iff$ `*(*(a+i)+j)`。

7.3 复杂指针声明解析

- **const 修饰**（“左定值，右定向”）：
  - `const int *p`：`*p` (内容) 不可变，p (指向) 可变。
  - `int * const p`：p (指向) 不可变，`*p` (内容) 可变。
- **指针数组 vs 数组指针**：
  - `int *p[10]`：**数组**，存了 10 个指针。
  - `int (*p)[10]`：**指针**，指向长度为 10 的数组。
- **函数指针**：
  - 定义：`int (*func)(int, int);`
  - 用途：回调函数（如 qsort 的比较函数）。

7.4 简答题预测

Q: 什么是指针？为什么要使用指针？

A: 指针是存储内存地址的变量。作用：1. 直接操作内存；2. 函数间传递大数据（避免拷贝）；3. 动态内存分配；4. 操作复杂数据结构。

8 函数与递归

- **传值 vs 传址**：修改实参需传指针。数组传参退化为指针。
- **递归 (Recursion)**：
  - **要素**：基准情况 (Base Case) + 递归步骤 (Recursive Step)。
  - **优缺点**：代码简洁但开销大（栈空间）。
- **存储类别**：
  - `auto`：栈内存。
  - `static`：延长局部变量生命周期；限制全局变量作用域。
  - **真题考点**：局部变量与全局变量同名时，局部变量优先（屏蔽全局变量）。

9 结构体、共用体与枚举

9.1 核心考点

- **结构体 (Struct)**：
  - 成员拥有独立内存。
  - **内存对齐**：总大小 $\geq$ 成员之和。
  - **真题考点**：`typedef struct {...} Name;` 定义别名，方便使用。
- **共用体 (Union)**：
  - 成员共享同一块内存。
  - 总大小 = 最大成员大小。
- **枚举 (Enum)**：默认从 0 开始递增。

9.2 简答题预测

Q: 简述结构体和共用体的区别。

A: 1. 内存：结构体成员独立，共用体成员共享。2. 大小：结构体累加（含对齐），共用体取最大。3. 赋值：共用体同一时间只能存一个值。

10 动态内存管理 (Dynamic Memory)

10.1 栈 (Stack) 与堆 (Heap) 的区别

- **栈 (Stack)：**
  - 由编译器自动分配释放。
  - 存放局部变量、函数参数。
  - 空间较小，速度快。
- **堆 (Heap)：**
  - 由程序员手动分配 (`malloc`) 和释放 (`free`)。
  - 空间大，但容易产生内存泄漏或碎片。

10.2 核心函数详解

- **malloc：** `void *malloc(size_t size);`
  - 分配指定字节数的内存。
  - **内容未初始化**（随机值）。
  - 返回 `void*`，通常需要强制类型转换（C 语言中可省略，但建议写上）。
  - **示例：** `int *p = (int*)malloc(n * sizeof(int));`
- **calloc：** `void *calloc(size_t n, size_t size);`
  - 分配 $n \times size$ 字节。
  - **自动初始化为 0**。
- **realloc：** `void *realloc(void *ptr, size_t new_size);`
  - 调整已分配内存的大小。
  - 可能移动内存块位置，必须接收返回值更新指针。
- **free：** `void free(void *ptr);`
  - 释放内存，归还给系统。
  - **注意：** 释放后指针变量本身的值未变（悬空指针），建议立即置为 `NULL`。

10.3 常见错误与最佳实践

- **忘记检查 NULL：**

```c
int *p = (int*)malloc(1000 * sizeof(int));
if (p == NULL) {
    // 处理分配失败
    exit(1);
}
```

```

```

- **内存泄漏 (Memory Leak)**：申请了内存但没有 free，导致程序占用内存越来越多。
- **悬空指针 (Dangling Pointer)**：使用已经 free 过的指针。
- **重复释放**：对同一个指针 free 两次（会导致崩溃）。

11 文件操作详解 (File I/O)

11.1 文件打开与关闭

- **打开文件**：FILE *fp = fopen("filename", "mode");
- **检查是否成功**：必须检查 fp 是否为 NULL。

```
    if ((fp = fopen("data.txt", "r")) == NULL) {
        printf("Cannot open file.\n");
        exit(1);
    }

```

- **关闭文件**：fclose(fp); 也就是保存文件，释放缓冲区。

11.2 文件打开模式 (Mode)

- "r"：**只读**。文件必须存在，否则报错 (NULL)。
- "w"：**只写**。文件不存在则创建；**存在则清空**（危险！）。
- "a"：**追加**。文件不存在则创建；存在则在末尾追加。
- "r+"：**读写**。文件必须存在。
- "w+"：**读写**。新建或清空。
- "a+"：**读写追加**。
- **二进制模式**：加 b (如 "rb", "wb")。区别在于换行符处理（Windows 下 \r\n ↔ \n）。

11.3 读写函数辨析

- **字符读写**：
  - ch = fgetc(fp)：读一个字符。结束返回 EOF。
  - fputc(ch, fp)：写一个字符。
- **字符串读写**：

  - `fgets(str, n, fp)`：读取一行（最多 $n-1$ 个字符），**保留换行符**，自动加 `\0`。
  - `fputs(str, fp)`：写入字符串，不会自动加换行符。
- **格式化读写**（文本文件）：
  - `fscanf(fp, "%d", &num)`：从文件读。
  - `fprintf(fp, "Result: %d", num)`：写入文件。
- **数据块读写**（二进制文件）：
  - `fread(buffer, size, count, fp)`:从文件读 count 个 size 大小的块到 buffer。
  - `fwrite(buffer, size, count, fp)`：将 buffer 中 count 个 size 大小的块写入文件。
  - **返回值**：成功读/写的块数 (`count`)。

11.4 文件定位与随机读写

- `rewind(fp)`：文件指针回到开头。
- `ftell(fp)`：返回当前文件指针位置（字节偏移量）。
- `fseek(fp, offset, origin)`：移动指针。
  - `SEEK_SET` (0)：文件头。
  - `SEEK_CUR` (1)：当前位置。
  - `SEEK_END` (2)：文件尾。

11.5 高频易错点 (坑)

- **feof 的误用**：
  - `while(!feof(fp))` 是错误的！
  - 原因：`feof` 只有在**读取失败**（读到了文件尾之后）才会返回真。这会导致最后一行数据被处理两次。
  - **正确写法**：检查读取函数的返回值。

```
        while (fscanf(fp, "%d", &n) != EOF) { ... }
        // 或者
        while (fgets(buf, 100, fp) != NULL) { ... }

```

- **路径分隔符**：Windows 路径 `C:\test.txt` 在字符串中要写成 `"C:\\test.txt"`。
- **读写同步**：在 `r+` 等读写模式下，读和写之间必须调用 `fseek` 或 `fflush` 刷新缓冲区，否则数据会错乱。

11.6 预处理指令补充

- **Include**：<> 查系统库，"" 查当前目录。

12 核心算法详解与易错点深度剖析

12.1 排序算法 (Sorting)

12.1.1 冒泡排序 (Bubble Sort)

核心思想：像水底的气泡一样，每一轮通过两两比较和交换，将当前未排序部分的最大值“浮”到数组的最右端。

```c
void bubble_sort(int arr[], int n) {
    int i, j, temp;
    // 外层循环：控制排序轮数，共需 n-1 轮
    for (i = 0; i < n - 1; i++) {
        // 内层循环：负责两两比较
        // 易错点1：边界是 n-1-i，因为最后 i 个元素已经排好序了
        for (j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) { // 升序用 >，降序用 <
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

易错点与考点：

- **内层循环边界**：`j < n - 1 - i`。
  - 如果写成 `j < n - 1`，程序也能跑，但效率低。
  - 如果写成 `j < n`，在访问 `arr[j+1]` 时会**数组越界**。
- **交换位置**：交换是在**内层循环**的 `if` 语句中发生的。
- **稳定性**：冒泡排序是**稳定**的。

12.1.2 选择排序 (Selection Sort)

核心思想：每一轮从未排序区域中找到**最小值**的下标，然后将其与未排序区域的第一个元素交换。

```c
void selection_sort(int arr[], int n) {
```

```c
    int i, j, min_idx, temp;
    // 外层循环：控制当前要确定的位置，从 0 到 n-2
    for (i = 0; i < n - 1; i++) {
        min_idx = i; // 假设当前位置 i 就是最小值

        // 内层循环：在剩余元素中找真正的最小值下标
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j; // 更新最小值下标，注意这里不交换！
            }
        }

        // 易错点2：交换发生在内层循环结束后
        if (min_idx != i) {
            temp = arr[i];
            arr[i] = arr[min_idx];
            arr[min_idx] = temp;
        }
    }
}
```

易错点与考点：

- **交换时机**：千万不要在内层循环里交换！内层循环只负责**找下标** (`min_idx`)。
- **比较对象**：内层循环是 `if (arr[j] < arr[min_idx])`，而不是 `arr[j] < arr[i]`。
- **稳定性**：选择排序是**不稳定**的。

12.1.3 插入排序 (Insertion Sort)

核心思想：像打扑克牌理牌一样，将新抓到的牌（`key`）插入到左手已经排好序的牌堆中的正确位置。

```c
void insertion_sort(int arr[], int n) {
    int i, j, key;
    // 从第2个元素开始（下标1），因为第1个元素默认有序
    for (i = 1; i < n; i++) {
        key = arr[i]; // 待插入的牌
        j = i - 1;    // 左边已排序区域的最后一个元素

        // 易错点3: while 循环条件
        // j >= 0 保证不越界
        // arr[j] > key 保证找到比 key 大的元素往后挪
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // 元素后移
```

```
            j--; // 继续向前找
        }
        arr[j + 1] = key; // 插入到正确位置
    }
}
```

易错点与考点：

- **后移操作：** `arr[j + 1] = arr[j]` 是覆盖操作，所以必须先保存 `key`。
- **插入位置：** 循环结束时，`key` 应该放在 `j + 1` 的位置。

12.2 查找算法 (Searching)

12.2.1 二分查找 (Binary Search)

前提条件： 数组必须是**有序**的。

```
int binary_search(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    int mid;

    while (low <= high) { // 易错点4: 是 <= 不是 <
        // 易错点5: 防溢出写法
        mid = low + (high - low) / 2;

        if (arr[mid] == key) {
            return mid; // 找到了
        } else if (arr[mid] < key) {
            low = mid + 1; // 在右半边, low 必须 +1
        } else {
            high = mid - 1; // 在左半边, high 必须 -1
        }
    }
    return -1; // 没找到
}
```

易错点与考点：

- **循环条件：** `while (low <= high)`。如果写成 `<`，会漏掉边界情况。
- **死循环陷阱：** 更新边界时必须是 `mid + 1` 或 `mid - 1`。
- **Mid 计算：** `(low + high) / 2` 可能溢出，推荐 `low + (high - low) / 2`。

12.3 数学与逻辑算法

12.3.1 辗转相除法 (GCD)

核心思想： gcd(a, b) = gcd(b, a % b)。

```
// 递归版
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
```

12.3.2 素数判断 (Prime Check)

核心思想： 除了 1 和它本身外，不能被其他数整除。

```
#include <math.h> // 需要用到 sqrt

int is_prime(int n) {
    if (n <= 1) return 0; // 易错点6: 1 不是素数

    // 优化: 只需要判断到 sqrt(n)
    int limit = (int)sqrt(n);
    for (int i = 2; i <= limit; i++) {
        if (n % i == 0) {
            return 0; // 发现因子, 不是素数
        }
    }
    return 1; // 是素数
}
```

易错点与考点：

- **特判 1：** 1 既不是质数也不是合数，必须单独判断返回 0。
- **循环终止条件：** i * i <= n 或 i <= sqrt(n)。

12.4 字符串处理算法

12.4.1 字符串逆序

核心思想： 双指针法，首尾交换，向中间逼近。

```
void reverse_string(char str[]) {
    int len = strlen(str);
    int i = 0;          // 头指针
    int j = len - 1;    // 尾指针
```

```
    char temp;

    while (i < j) {
        temp = str[i];
        str[i] = str[j];
        str[j] = temp;
        i++;
        j--;
    }
}
```

易错点与考点：

- **尾指针位置：** `len - 1`。
- **循环条件：** `i < j`。
- **不要动 \0：** `\0` 是字符串结束符，不参与交换。

12.5 考试手写代码“救命”技巧

1. **头文件:**用到 `printf` 写 `#include <stdio.h>`,用到 `malloc` 写 `#include <stdlib.h>`，用到 `strlen` 写 `#include <string.h>`，用到 `sqrt` 写 `#include <math.h>`。
2. **变量初始化：** 计数器 `count`、累加器 `sum` 一定要初始化为 0。
3. **分号检查：** `if`、`for`、`while` 后面千万别手顺加分号！
4. **返回值：** `main` 函数最后记得写 `return 0;`。

13 C 语言易混淆概念辨析

- **赋值 vs 相等：**
  - `=` 是赋值运算符（把右边的值给左边）。
  - `==` 是关系运算符（判断左右两边是否相等）。
  - **陷阱：** `if(x=5)` 永远为真，`x` 变为 5；`if(x==5)` 才是判断。
- **字符 vs 字符串：**
  - `'a'`：字符常量，占 1 字节。
  - `"a"`：字符串常量，包含 `'a'` 和 `'\0'`，占 2 字节。
- **逻辑与 vs 按位与：**
  - `&&` (逻辑与)：短路特性，结果为 0 或 1。
  - `&` (按位与)：按二进制位操作，如 `3 & 5` ($011 \& 101 = 001$)。
- **sizeof vs strlen：**

  - `sizeof`：运算符，编译时计算，统计类型或变量占用的**内存字节数**（包含 `\0`）。
  - `strlen`：函数，运行时计算，统计字符串的**有效字符长度**（遇到 `\0` 停止，不含 `\0`）。
- **指针数组 vs 数组指针：**
  - `int *p[10]`：**数组**，里面存了 10 个 `int*` 指针。
  - `int (*p)[10]`：**指针**，指向一个包含 10 个 `int` 的数组。
- **p++ vs (*p)++：**
  - `p++`：指针向后移动一个单位（地址改变）。
  - `(*p)++`：指针指向的值加 1（地址不变，内容改变）。
- **break vs continue：**
  - `break`：彻底跳出当前循环（或 switch）。
  - `continue`：跳过本次循环剩余语句，直接进入下一次循环判断。
- **形参 vs 实参：**
  - **单向传递**：实参的值传给形参，形参改变不影响实参（除非传指针）。

14 冷门与进阶考点拾遗

- **逗号运算符：**
  - 优先级最低。
  - 表达式的值是**最后一个表达式**的值。
  - 例：`a = 3, 4, 5;` → `a` 为 3（赋值优先级高于逗号）。
  - 例：`a = (3, 4, 5);` → `a` 为 5。
- **宏定义陷阱：**
  - 宏只是简单的文本替换，不进行计算。
  - 例：`#define S(r) r*r`。若调用 `S(a+b)`，展开为 `a+b*a+b`，而非 `(a+b)*(a+b)`。
  - **对策**：宏定义中每个参数都要加括号，整体也要加括号。
- **转义字符：**
  - `\ddd`：1-3 位八进制数代表的字符（如 `\101` 是'A'）。
  - `\xhh`：1-2 位十六进制数代表的字符。
  - 注意：`\08` 是非法的（八进制没有 8）。
- **格式化输出细节：**
  - `%5d`：域宽为 5，右对齐。
  - `%-5d`：域宽为 5，左对齐。
  - `%05d`：域宽为 5，不足补 0。

  - `%5.2f`：总宽 5，小数点后 2 位。
- **运算符优先级口诀：**
  - **单目** (!, ++, –, sizeof) > **算术** (*, /, %) > **算术** (+, -) > **移位** («, ») > **关系** (>, <) > **相等** (==, !=) > **位逻辑** (&, ^, |) > **逻辑** (&&, ||) > **三目** (? :) > **赋值** (=) > **逗号** (,)。
  - 记不住就加括号！
- **typedef 与 #define 的区别：**
  - `#define` 是预处理指令，仅做文本替换。
  - `typedef` 是编译指令，为类型创建别名，有作用域检查。
  - 例：`typedef int* ptr; ptr a, b;` → a, b 都是指针。
  - 例：`#define PTR int* PTR a, b;` → a 是指针，b 是整型（替换后为 `int * a, b;`）。

15 简答题预测汇总

以下是根据历年考点和课程核心内容整理的简答题题库，建议背诵关键词。

15.1 计算机基础与算法

Q1: 简述冯·诺依曼体系结构的核心思想（“存储程序”原理）。

A: 将程序（指令序列）和数据一起存储在计算机的存储器中，计算机在运行时自动地从存储器中取出指令并执行，而不需要人工干预。

Q2: 什么是算法？它有哪些基本特征？

A: 算法是解决特定问题的一系列明确、有限的指令。**五大特征：**

1. **有穷性：** 步骤有限，在有限时间内完成。
2. **确定性：** 每一步指令清晰无歧义。
3. **可行性：** 每一步都能通过基本运算实现。
4. **输入：** 0 个或多个输入。
5. **输出：** 1 个或多个输出。

Q3: 结构化程序设计的三种基本控制结构是什么？

A: 顺序结构、选择结构（分支结构）、循环结构。

15.2 C 语言核心概念

Q4: 什么是指针？为什么要使用指针？

A: 指针是存储内存地址的变量。**作用：**

1. 可以直接操作内存地址，提高程序效率。
2. 实现函数间的大数据传递（避免拷贝整个数组或结构体）。
3. 支持动态内存分配（malloc/free）。
4. 处理复杂的数据结构（如链表、树）。

Q5: 简述递归函数的优缺点。

A: 优点：代码简洁，逻辑清晰，适合解决具有重复子结构的问题（如汉诺塔、树的遍历）。**缺点：**函数调用开销大，占用栈空间多，深度过深可能导致栈溢出，效率通常低于迭代。

Q6: 结构体 (struct) 和共用体 (union) 的区别是什么？

A: 结构体：每个成员拥有独立的内存空间，总大小大于等于所有成员大小之和（受内存对齐影响）。所有成员可以同时存在。**共用体：**所有成员共享同一块内存空间，总大小等于最大成员的大小。同一时刻只能存储其中一个成员的值。

Q7: 局部变量和全局变量的区别？

A: 作用域：局部变量仅在定义它的函数或代码块内有效；全局变量在整个源程序文件中有效。**生命周期：**局部变量随函数调用创建，函数结束销毁（static 除外）；全局变量在程序运行期间一直存在。**优先级：**同名时，局部变量屏蔽全局变量。

Q8: 关键字 static 的作用有哪些？

A:

1. **修饰局部变量：**改变生命周期，使其在程序运行期间一直存在，但作用域不变（仍局限于函数内）。
2. **修饰全局变量/函数：**限制作用域，使其仅在当前源文件中可见，不能被其他文件引用（隐藏）。

15.3 常见算法特点与比较

Q9: 简述三种基本排序算法（冒泡、选择、插入）的区别与特点。

A:

- **冒泡排序：**两两比较相邻元素，大数下沉。
  - 最好 $O(n)$（已有序），最坏 $O(n^2)$。

  - **稳定**。
- **选择排序**：每次从未排序区选最小的放到已排序区末尾。
  - 无论是否有序，复杂度恒为 $O(n^2)$。
  - **不稳定**（如序列 5, 8, 5, 2，第一趟交换第一个 5 和 2，导致两个 5 相对位置改变）。
- **插入排序**：将新元素插入到已排序序列的合适位置。
  - 最好 $O(n)$，最坏 $O(n^2)$。对于**基本有序**的数据效率最高。
  - **稳定**。

Q10: 顺序查找与二分查找的对比。

A:

- **顺序查找**：
  - 适用性：无序或有序数组均可，链表也可。
  - 复杂度：$O(n)$。
- **二分查找**：
  - 适用性：**必须是有序数组**（支持随机访问）。
  - 复杂度：$O(\log_2 n)$。效率远高于顺序查找。

Q11: 什么是算法的时间复杂度和空间复杂度？

A:

- **时间复杂度**：算法执行所需时间与问题规模 n 之间的增长关系（通常用大 O 表示法，如 $O(n), O(n^2)$）。衡量算法运行快慢。
- **空间复杂度**：算法执行过程中所需的存储空间与问题规模 n 之间的增长关系。衡量算法占用内存多少。

15.4 实验课补充简答题

Q12: 有序数组与无序数组在操作效率上有何区别？

A:

- **无序数组**：插入最快 $O(1)$（直接追加），查找和删除较慢 $O(n)$（线性查找）。
- **有序数组**：查找最快 $O(\log n)$（二分查找），插入和删除较慢 $O(n)$（需要移动元素保持有序）。

Q13: 栈 (Stack) 的主要特性和基本操作有哪些？

A:

- **特性**：后进先出 (LIFO, Last In First Out)。
- **基本操作**：
  - **Push (入栈)**：将元素放入栈顶。
  - **Pop (出栈)**：移除并返回栈顶元素。
  - **Peek (查看栈顶)**：返回栈顶元素但不移除。
  - **IsEmpty (判空)**：检查栈是否为空。

Q14: 如何利用栈实现括号匹配（如 JSON 字符串校验）?

A: 遍历字符串，遇到左括号（如 `{`，`[`）入栈；遇到右括号（如 `}`，`]`）时，检查栈顶元素是否为对应的左括号。若是，则弹出栈顶元素继续；若否或栈为空，则匹配失败。遍历结束后栈应为空。

Q15: 在“找出兴趣值最大的 k 个客户”问题中，为什么不需要全排序?

A: 全排序的时间复杂度通常为 $O(n \log n)$ 或 $O(n^2)$。如果只需前 k 个最大值，可以使用**部分选择排序**（执行 k 轮选择），时间复杂度为 $O(k \cdot n)$。当 $k \ll n$ 时，效率更高。

Q16: 递归解决汉诺塔问题的核心逻辑是什么?

A: 将 n 个盘子从 A 移到 C 的过程分解为三步：

1. 将 $n-1$ 个盘子从 A 移到 B（借助 C）。
2. 将第 n 个盘子（最大的）从 A 移到 C。
3. 将 $n-1$ 个盘子从 B 移到 C（借助 A）。

这是一个典型的分治递归思想。